

PROPOSAL PROYEK ALGORITMA DAN PEMROGRAMAN KELOMPOK 4



JUDUL

**Implementasi Algoritma Searching dan Sorting pada Sistem Manajemen
Inventaris dan Toko Jallaludin**

KELOMPOK 4 :

1. HAYDEN PUTRA PRADANA M - 252410103040
2. VEDA BASTIAN AR RAMZY - 252410103037
3. ALTHA LINGGA LEKSONO - 252410103068
4. DAVA JULIAN EKA PUTRA - 252410103074

DOSEN PENGAMPU

**Rizky Alfania Atmoko S.Si. M.Sc.
Fadhel Akhmad Hizham S.Kom., M.Kom.**

**FAKULTAS ILMU KOMPUTER
UNIVERSITAS JEMBER**

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan penyusunan laporan yang berjudul "Implementasi Algoritma Searching dan Sorting pada Sistem Manajemen Inventaris dan Toko Jallaludin" ini dengan baik dan tepat pada waktunya.

Dalam proses penyusunannya, Kami menyadari sepenuhnya bahwa makalah ini masih jauh dari kata sempurna, baik dari segi materi, penyusunan, maupun penggunaan kata. Oleh karena itu, kami sangat terbuka terhadap kritik serta saran yang membangun dari para pembaca. Masukan tersebut akan sangat berguna bagi kami untuk memperbaiki kualitas tulisan di masa yang akan datang.

Akhir kata, kami berharap semoga makalah ini dapat memberikan manfaat, inspirasi, serta tambahan informasi bagi kita semua.

Jember, ... Mei 2026

Penulis

DAFTAR ISI

COVER.....	1
KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I.....	5
PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Rumusan Masalah.....	5
1.3 Tujuan.....	6
BAB II.....	7
LANDASAN TEORI.....	7
2.1 Insertion Sort.....	7
2.2 Binary Search.....	7
2.3 Flowchart.....	8
BAB III.....	9
PERANCANGAN PROGRAM.....	9
3.1 Spesifikasi Kebutuhan Sistem.....	9
3.2 Arsitektur dan Manajemen Data.....	9
3.3 Implementasi Algoritma.....	9
3.4 Roadmap Pengembangan dan Mitigasi Risiko.....	10
3.5 Fitur.....	10
3.5.1 Tambah Barang.....	10
3.5.2 Lihat Semua Inventaris.....	13
3.5.3 Cari Barang Dalam Inventaris.....	14
3.5.4 Hapus Barang Dari Inventaris.....	15
3.5.5 Update Barang Inventaris.....	16
3.5.6 Simpan Data.....	18
BAB IV.....	19
HASIL.....	19
4.1 Menu Utama.....	19
4.2 Kesesuaian dengan Materi Kuliah Algoritma dan Pemrograman.....	21
DAFTAR PUSTAKA.....	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan inventaris merupakan tulang punggung bagi keberlangsungan sebuah usaha retail. Namun, pada prakteknya Toko Jallaludin masih menggunakan metode pencatatan manual dalam buku besar. Seiring dengan bertambahnya varietas barang, metode manual ini mulai menimbulkan berbagai masalah yang menghambat efisiensi pendataan toko.

Masalah pertama muncul pada bagian manajemen stok. Tanpa sistem digital yang terintegrasi pemilik toko seringkali kesulitan untuk memantau jumlah stok. Hal ini menyebabkan ketidakteraturan pendataan dimana barang yang kosong susah terdeteksi atau justru terjadi penumpukan barang yang tidak laku. Kondisi ini diperparah saat proses pencarian barang dalam daftar fisik yang memakan waktu lama sehingga pengelolaan data barang menjadi tidak optimal.

Masalah kedua terkait dengan pengelolaan berkas. Pencatatan data harian yang dilakukan secara manual sangat rentan terhadap kesalahan, mulai dari salah tulis jumlah stok hingga hilangnya nota fisik. Akibatnya, pemilik toko kesulitan untuk mengetahui jumlah stok secara akurat, melacak alur masuk-keluar barang, serta menyusun laporan inventaris bulanan yang valid. Tanpa data inventaris yang akurat pengambilan keputusan strategis untuk pengembangan toko menjadi sulit dilakukan.

Dari permasalahan tersebut, diperlukan sebuah solusi digital berupa aplikasi Sistem Manajemen Inventaris. Sistem ini akan mengimplementasikan algoritma Insertion Sort untuk merapikan pendataan barang secara otomatis dan Binary Search untuk mempercepat pencarian data dalam jumlah besar. Dengan beralih dari sistem manual ke sistem digital yang terotomatisasi, Toko Jallaludin dapat meminimalisir resiko kehilangan data, mempercepat pencarian informasi barang, dan memiliki pendataan yang transparan serta akuntabel.

1.2 Rumusan Masalah

1. Bagaimana mengatasi ketidakteraturan pendataan inventaris barang yang selama ini masih dilakukan secara manual di Toko Jallaludin?
2. Bagaimana merancang sistem pencarian barang yang cepat dan akurat agar kasir tidak kesulitan saat melayani pelanggan di tengah ribuan data stok?
3. Bagaimana cara memberikan kemudahan bagi pemilik toko dalam memantau keuangan dan stok barang secara *real-time*?

1.3 Tujuan

1. Menggantikan sistem manual dengan basis data digital yang terstruktur guna mengatasi ketidakteraturan pendataan inventaris di Toko Jallaludin.
2. Merancang dan membangun fitur pencarian barang yang responsif dan akurat untuk mempermudah operasional kasir dalam menemukan informasi produk di tengah database yang besar.
3. Mengimplementasikan algoritma Insertion Sort dan Binary Search ke dalam sistem untuk membuktikan peningkatan efisiensi waktu dalam pengurutan dan pencarian data inventaris.

BAB II

LANDASAN TEORI

2.1 Insertion Sort

Insertion Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara membandingkan setiap elemen data satu per satu, kemudian menyisipkannya (*insert*) ke posisi yang tepat dalam daftar yang sudah terurut. Logika algoritma ini sering dianalogikan dengan cara seseorang mengurutkan kartu di tangan, di mana satu per satu kartu diambil dan diletakkan pada posisi yang sesuai dibandingkan kartu lainnya.

Alasan Pemilihan:

- **Prasyarat Binary Search:** Pengurutan menggunakan *Insertion Sort* menjadi pondasi utama agar fitur pencarian cepat menggunakan Binary Search dapat berfungsi dengan akurat dalam mencari stok barang.
- **Kemudahan Implementasi:** Algoritma ini memiliki logika yang sederhana dan mudah diimplementasikan dalam struktur data *array* maupun *vector*, sehingga sangat cocok digunakan untuk pengembangan sistem manajemen tahap awal.
- **Kinerja Optimal pada Data Hampir Terurut:** Dalam sistem inventaris, seringkali data barang hanya mengalami sedikit perubahan (seperti penambahan satu atau dua item baru). *Insertion Sort* sangat cepat dalam menangani data yang sudah hampir terurut dibandingkan algoritma kompleks lainnya.

2.2 Binary Search

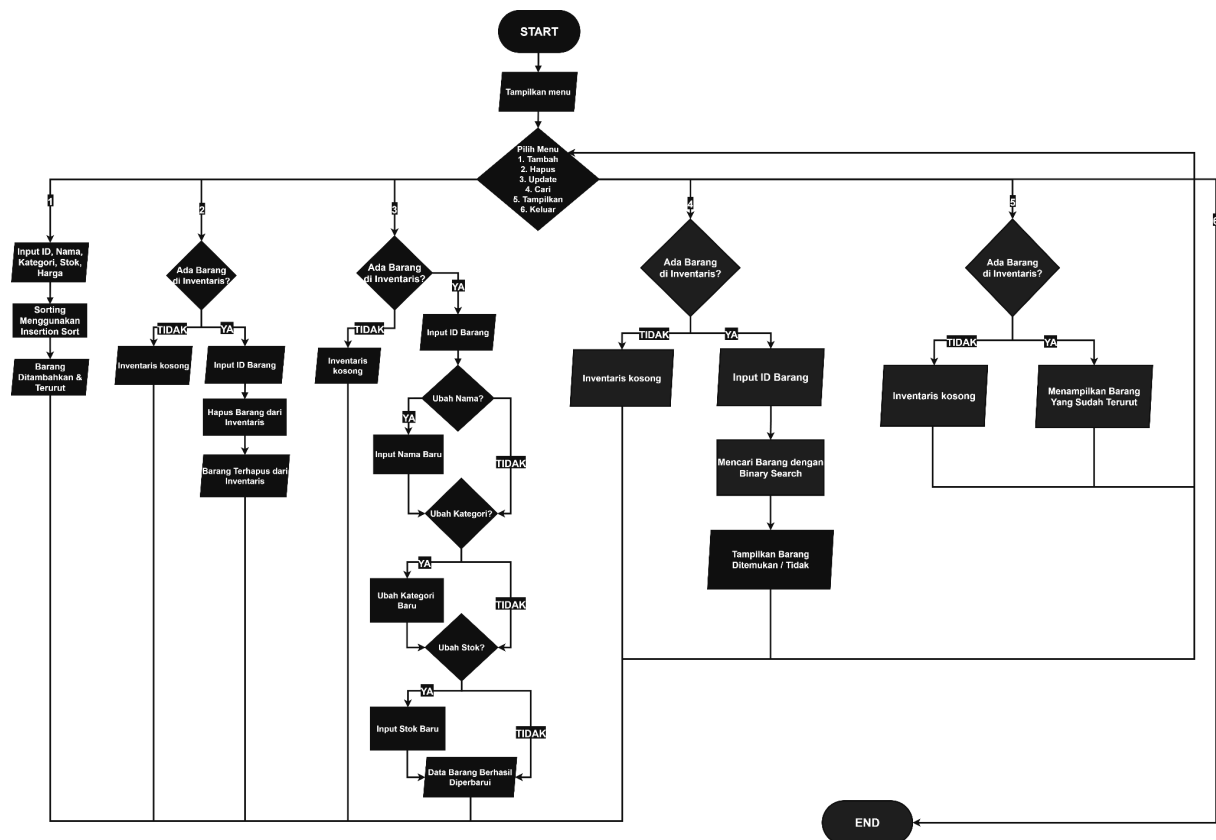
Binary Search adalah metode pencarian pada data yang telah terurut dengan cara membagi dua ruang pencarian secara berulang. Berbeda dengan pencarian linear yang memeriksa data satu per satu dari awal, Binary Search langsung memeriksa nilai tengah. Jika data yang dicari lebih kecil dari nilai tengah, pencarian dilanjutkan pada belahan kiri, dan sebaliknya.

Alasan Pemilihan:

- **Efisiensi Waktu:** Proses pencarian sangat singkat. Untuk mencari satu barang di antara 100 item, sistem hanya memerlukan maksimal 10 langkah pengecekan.
- **Optimalisasi Pelayanan:** Mempercepat kerja kasir saat melayani pembeli, terutama pada jam operasional yang padat.

2.3 Flowchart

Flowchart berikut menggambarkan bagaimana data inventaris diproses di dalam sistem, mulai dari pengurutan otomatis hingga proses pencarian barang.



BAB III

PERANCANGAN PROGRAM

3.1 Spesifikasi Kebutuhan Sistem

Bagian ini mendefinisikan batasan teknis dan fungsionalitas utama yang berfokus pada digitalisasi manajemen stok di Toko Jallaludin:

- **Kebutuhan Fungsional:** Sistem wajib mampu mengelola minimal 500 records data inventaris yang mencakup informasi ID Barang, Nama Barang, dan Harga Barang.
- **Antarmuka Pengguna (GUI):** Sistem akan diimplementasikan menggunakan antarmuka pengguna grafis (GUI) dengan navigasi menu.
- **Prioritas Pengembangan:** Fokus utama pengembangan sistem pada tahap awal ini sepenuhnya diarahkan pada inventaris (manajemen stok barang). Langkah ini diambil agar tim dapat berfokus penuh pada optimasi antarmuka GUI serta akurasi implementasi algoritma pencarian dan pengurutan pada data barang.

3.2 Arsitektur dan Manajemen Data

Aplikasi ini dibangun menggunakan bahasa pemrograman C++ dengan spesifikasi teknis sebagai berikut:

- **Struktur Data:** Informasi barang dibungkus dalam tipe data “Struct,” dan dikelola menggunakan “vector,” untuk memberikan fleksibilitas pada memori saat terjadi penambahan data secara dinamis.
- **Penyimpanan Data:** Menggunakan pustaka “<fstream>,” untuk sinkronisasi data dengan file.csv Sistem akan membaca data saat aplikasi dimulai dan memperbarui file tersebut saat aplikasi ditutup.

3.3 Implementasi Algoritma

Pemilihan algoritma didasarkan pada frekuensi penggunaan sistem yang lebih sering melakukan pencarian barang dibandingkan penginputan barang baru:

- **Insertion Sort:** Algoritma ini digunakan untuk menjaga data tetap terurut berdasarkan ID Barang setiap kali ada input data masuk. Pemilihan *Insertion Sort* didasarkan pada efisiensinya dalam menangani data yang "hampir terurut," yang sangat cocok dengan mekanisme input barang secara berkala.
- **Binary Search:** Mengingat aktivitas pencarian barang di antara ribuan data adalah yang paling sering dilakukan, algoritma *Binary Search* digunakan untuk memberikan hasil pencarian yang instan berdasarkan ID Barang.

3.4 Roadmap Pengembangan dan Mitigasi Risiko

Pengembangan dilakukan melalui tahapan yang sistematis untuk menjamin integritas data:

- **Tahap 1 (MVP):** Pengembangan menu utama GUI, integrasi File I/O untuk 100 data dummy, dan fungsi pencarian *Binary Search*.
- **Tahap 2 (Optimasi):** Implementasi *Insertion Sort* pada menu "Tambah Barang," untuk memastikan data selalu siap untuk diproses oleh fungsi pencarian.
- **Mitigasi Risiko:** Validasi tipe data pada file CSV (misalnya memastikan harga tetap integer) dilakukan secara ketat untuk mencegah *crash* atau *infinite loop* pada program C++.

3.5 Fitur

3.5.1 Tambah Barang

Fitur ini digunakan ketika pengguna untuk memasukkan data barang ke dalam Inventaris. Fitur ini menggunakan Algoritma Insertion Sort untuk mengurutkan list inventaris setelah menambahkan baru ke dalam inventaris.

```
void TambahBarang() {
    Barang barangBaru;

    cout << "\n=== TAMBAH BARANG MASUK ===" << endl;
    while (true) {
        cout << "Masukkan ID Barang (contoh: BRG025): ";
        getline(cin, barangBaru.ID_Barang);
        if (barangBaru.ID_Barang.empty()) {
            cout << "ERROR: ID Barang tidak boleh kosong!" << endl;
        } else if (barangBaru.ID_Barang.find(',') != string::npos) {
            cout << "ERROR: ID Barang tidak boleh mengandung karakter koma (,)" << endl;
        } else {
            break;
        }
    }

    for (const auto& item : inventaris) {
        if (item.ID_Barang == barangBaru.ID_Barang) {
            cout << "[ERROR]: ID Barang sudah ada di database! Gagal menambahkan." << endl;
            return;
        }
    }

    while (true) {
        cout << "Masukkan Nama Barang: ";
        getline(cin, barangBaru>Nama_Barang);
        if (barangBaru>Nama_Barang.empty()) {
            cout << "[ERROR]: Nama Barang tidak boleh kosong!" << endl;
        } else if (barangBaru>Nama_Barang.find(',') != string::npos) {
            cout << "[ERROR]: Nama Barang tidak boleh mengandung karakter koma (,)" << endl;
        } else {
            break;
        }
    }
}
```

```

while (true) {
    cout << "Masukkan Kategori   : ";
    getline(cin, barangBaru.Kategori);
    if (barangBaru.Kategori.empty()) {
        cout << "[ERROR]: Kategori tidak boleh kosong!" << endl;
    } else if (barangBaru.Kategori.find(',') != string::npos) {
        cout << "[ERROR]: Kategori tidak boleh mengandung karakter koma (,)" << endl;
    } else {
        break;
    }
}

string stokInput;
bool stokValid = false;
while (!stokValid) {
    cout << "Masukkan Jumlah Stok (Masukkan Angka Saja, Misal 15): ";
    getline(cin, stokInput);
    string stokBersih = "";

    for (char c : stokInput) {
        if (isdigit(c)) stokBersih += c;
    }

    if (stokBersih.empty()) {
        cout << "[ERROR]: Gunakan Angka!\n";
    } else {
        try {
            barangBaru.Stok = stoi(stokBersih);
            stokValid = true;
        } catch (...) {
            cout << "[ERROR]: Jumlah stoknya terlalu banyak!\n";
        }
    }
}

```

```

string hargaInput;
bool hargaValid = false;

while (!hargaValid) {
    cout << "Masukkan Harga Barang (Boleh pakai titik/koma, misal 25.000): ";
    getline(cin, hargaInput);

    string hargaBersih = "";

    for (char c : hargaInput) {
        if (isdigit(c)) {
            hargaBersih += c;
        }
    }

    if (hargaBersih.empty()) {
        cout << "[ERROR]: Gunakan Angka!\n";
    } else {
        try {
            barangBaru.Harga_Barang = stoi(hargaBersih);
            hargaValid = true;
        } catch (...) {
            cout << "[ERROR]: Angka terlalu besar!\n";
        }
    }
}

inventaris.push_back(barangBaru);

UrutkanInventaris();

cout << "[SYSTEM]: Barang berhasil ditambahkan dan database telah diurutkan ulang!" << endl;
}

```

3.5.2 Lihat Semua Inventaris

Fitur ini digunakan untuk menampilkan keseluruhan data barang yang ada dalam Inventaris.

```
void TampilkanInventaris() {  
    if (inventaris.empty()) {  
        cout << "Inventaris kosong" << endl;  
        return;  
    }  
    cout << "\n=== DAFTAR BARANG TOKO JALLALUDIN ===" << endl;  
    for (const auto& item : inventaris) {  
        cout << "ID: " << item.ID_Barang  
            << " | Nama: " << item>Nama_Barang  
            << " | Kategori: " << item.Kategori  
            << " | Stok: " << item.Stok << " pcs"  
            << " | Harga: " << FormatRupiah(item.Harga_Barang) << endl;  
    }  
}
```

3.5.3 Cari Barang Dalam Inventaris

Fitur ini digunakan saat pengguna ingin mencari data suatu barang yang berada di dalam Inventaris. Fitur ini menggunakan Algoritma Binary Search untuk mencari data barang yang dicari.

```
void CariBarang() {
    if (inventaris.empty()) {
        cout << "[ERROR]: Inventaris kosong!" << endl;
        return;
    }

    string targetID;
    cout << "\n=== CARI BARANG ===" << endl;
    cout << "Masukkan ID Barang (contoh: BRG50): ";
    getline(cin, targetID);

    int kiri = 0;
    int kanan = (int)inventaris.size() - 1;
    bool ketemu = false;

    while (kiri <= kanan) {
        int tengah = kiri + (kanan - kiri) / 2;

        if (inventaris[tengah].ID_Barang == targetID) {
            cout << "\n[SYSTEM]: Barang Ditemukan!" << endl;
            cout << "ID          : " << inventaris[tengah].ID_Barang << endl;
            cout << "Nama       : " << inventaris[tengah].Nama_Barang << endl;
            cout << "Kategori  : " << inventaris[tengah].Kategori << endl;
            cout << "Stok      : " << inventaris[tengah].Stok << " pcs" << endl;
            cout << "Harga     : " << FormatRupiah(inventaris[tengah].Harga_Barang) << endl;
            ketemu = true;
            break;
        }

        else if (Banding(inventaris[tengah].ID_Barang, targetID)) {
            kiri = tengah + 1;
        }
        else {
            kanan = tengah - 1;
        }
    }

    if (!ketemu) {
        cout << "\n[ERROR]: Barang dengan ID '" << targetID << "' tidak ditemukan!" << endl;
    }
}
```

3.5.4 Hapus Barang Dari Inventaris

Fitur ini digunakan saat pengguna ingin menghapus data suatu barang yang berada di dalam Inventaris. Fitur ini berguna jika barang dalam inventaris sudah tidak layak untuk di distribusikan atau dijual.

```
void HapusBarang() {
    if (inventaris.empty()) {
        cout << "[ERROR]: Inventaris kosong, tidak ada yang dihapus!" << endl;
        return;
    }

    string targetID;
    cout << "\n=== HAPUS BARANG ===" << endl;
    cout << "Masukkan ID Barang (contoh: BRG50): ";
    getline (cin, targetID);

    std::string Barang::ID_Barang;
    bool ketemu;
    for (auto it << Generate Copilot summary << endl; ++it) {
        if (it->ID_Barang == targetID) {
            cout << "[SISTEM]: Barang '" << it->Nama_Barang << "' (Harga: " << FormatRupiah(it->Harga_Barang) << ") telah dihapus!" << endl;
            inventaris.erase(it);
            ketemu = true;
            break;
        }
    }

    if (!ketemu) {
        cout << "\n[ERROR]: Barang dengan ID '" << targetID << "' tidak ditemukan!" << endl;
    }
}
```

3.5.5 Update Barang Inventaris

Fitur ini digunakan saat terdapat data yang ingin diubah di data Inventaris.

```
void UbahDataBarang() {
    if (inventaris.empty()) {
        cout << "[ERROR]: Inventaris kosong, tidak ada yang bisa diubah!" << endl;
        return;
    }

    string targetID;
    cout << "\n=== UBAH DATA BARANG ===" << endl;
    cout << "Masukkan ID Barang yang ingin diubah (contoh: BRG50): ";
    getline(cin, targetID);

    auto it = find_if(inventaris.begin(), inventaris.end(), [&](const Barang& b) {
        return b.ID_Barang == targetID;
    });

    if (it == inventaris.end()) {
        cout << "\n[ERROR]: Barang dengan ID '" << targetID << "' tidak ditemukan!" << endl;
        return;
    }

    cout << "Masukkan Nama Baru (kosongkan jika tidak ingin mengubah): ";
    string namaBaru;
    while (true) {
        getline(cin, namaBaru);
        if (namaBaru.find(',') != string::npos) {
            cout << "[ERROR]: Nama tidak boleh mengandung karakter koma (!) Masukkan lagi: ";
        } else {
            break;
        }
    }
    if (!namaBaru.empty()) it->Nama_Barang = namaBaru;

    cout << "Masukkan Kategori Baru (kosongkan jika tidak ingin mengubah): ";
    string kategoriBaru;
    while (true) {
        getline(cin, kategoriBaru);
        if (kategoriBaru.find(',') != string::npos) {
            cout << "[ERROR]: Kategori tidak boleh mengandung karakter koma (!) Masukkan lagi: ";
        } else {
            break;
        }
    }
}
```



```

while (true) {
}
if (!kategoriBaru.empty()) it->Kategori = kategoriBaru;

string stokInput;
cout << "Masukkan Jumlah Stok Baru (kosongkan jika tidak ingin mengubah): ";
getline(cin, stokInput);
if (!stokInput.empty()) {
    string stokBersih = "";
    for (char c : stokInput) {
        if (isdigit(c)) stokBersih += c;
    }
    if (!stokBersih.empty()) {
        try {
            it->Stok = stoi(stokBersih);
        } catch (...) {
            cout << "[ERROR]: Jumlah stoknya terlalu banyak! Stok tidak diubah.\n";
        }
    } else {
        cout << "[ERROR]: Gunakan Angka! Stok tidak diubah.\n";
    }
}

string hargaInput;
cout << "Masukkan Harga Baru (kosongkan jika tidak ingin mengubah): ";
getline(cin, hargaInput);
if (!hargaInput.empty()) {
    string hargaBersih = "";
    for (char c : hargaInput) {
        if (isdigit(c)) hargaBersih += c;
    }
    if (!hargaBersih.empty()) {
        try {
            it->Harga_Barang = stoi(hargaBersih);
        } catch (...) {
            cout << "[ERROR]: Angka terlalu besar! Harga tidak diubah.\n";
        }
    } else {
        cout << "[ERROR]: Gunakan Angka! Harga tidak diubah.\n";
    }
}
}

```

```

    cout << "[SYSTEM]: Data barang dengan ID '" << targetID << "' berhasil diperbarui!" << endl;
}

```

3.5.6 Simpan Data

Fitur simpan data.

```
void BacaDataCSV() {
    ifstream file(namaFile);
    string baris, id, nama, kategori, stok_str, harga_str;

    if (!file.is_open() || file.peek() == std::ifstream::traits_type::eof()) {
        if (file.is_open()) file.close();

        cout << "[WARNING]: File " << namaFile << " tidak ditemukan atau kosong!" << endl;
        cout << "[SYSTEM]: Membuat File Inventaris Toko ..." << endl;

        ofstream fileBaru(namaFile);
        cout << "[SYSTEM]: File Inventaris Toko berhasil dibuat!\n" << endl;

        file.open(namaFile);
    }

    while (getline(file, baris)) {
        if (!baris.empty() && baris.back() == '\r') {
            baris.pop_back();
        }
        stringstream ss(baris);
        getline(ss, id, ',');
        getline(ss, nama, ',');
        getline(ss, kategori, ',');
        getline(ss, stok_str, ',');
        getline(ss, harga_str, ',');

        if (!id.empty() && !nama.empty() && !kategori.empty() && !stok_str.empty() && !harga_str.empty()) {
            Barang barangBaru;
            barangBaru.ID_Barang = id;
            barangBaru>Nama_Barang = nama;
            barangBaru.Kategori = kategori;
            barangBaru.Stok = (stok_str.find_first_not_of("0123456789") == string::npos) ? stoi(stok_str) : 0;
            barangBaru.Harga_Barang = (harga_str.find_first_not_of("0123456789") == string::npos) ? stoi(harga_str) : 0;
            inventaris.push_back(barangBaru);
        }
    }
    file.close();

    UrutkanInventaris();
}
```


BAB IV

HASIL

4.1 Menu Utama

Ketika aplikasi dijalankan akan tampil menu seperti pada gambar dibawah, terdapat 5 menu utama, yaitu Tambah Data Barang, Lihat semua Inventory, Cari Barang, dan Hapus Barang.

SIJALAL

Sistem Inventaris Jalaludin

id barang

nama barang

kategori

jumlah

harga

perbarui

tambah

cari

hapus

RESET

	ID Barang	Nama Barang	Kategori	Jum
▶	BRG1	Barang_Toko_1	Random1	4200
	BRG2	Barang_Toko_2	Random2	3350
	BRG3	Barang_Toko_3	Random3	1700
	BRG4	Barang_Toko_4	Random4	4790
	BRG5	Barang_Toko_5	Random5	9630
	BRG6	Barang_Toko_6	Random6	7060
	BRG7	Barang_Toko_7	Random7	2820
	BRG8	Barang_Toko_8	Random8	9620
	BRG9	Barang_Toko_9	Random9	9960
	BRG10	Barang_Toko_10	Random10	8280
	BRG11	Barang_Toko_11	Random11	3920
	BRG12	Barang_Toko_12	Random12	9030

Gambar 4.1 Menu Utama

SIJALAL

Sistem Inventaris Jalaludin

id barang

nama barang

kategori

jumlah

harga

perbarui

tambah

cari

hapus

RESET

	ID Barang	Nama Barang	Kategori	Jum
	BRG92	Barang_Toko_92	Random92	5400
	BRG93	Barang_Toko_93	Random93	4190
	BRG94	Barang_Toko_94	Random94	9010
	BRG95	Barang_Toko_95	Random95	1280
	BRG96	Barang_Toko_96	Random96	7290
	BRG97	Barang_Toko_97	Random97	6490
	BRG98	Barang_Toko_98	Random98	8080
	BRG99	Barang_Toko_99	Random99	3110
	BRG100	Barang_Toko_100	Random100	8140
	BRG101	Keju	Makanan	1
*				

Gambar 4.2 Menu Tambah Barang

	ID Barang	Nama Barang	Kategori	Jum
►	BRG1	Barang_Toko_1	Random1	4200
	BRG2	Barang_Toko_2	Random2	3350
	BRG3	Barang_Toko_3	Random3	1700
	BRG4	Barang_Toko_4	Random4	4790
	BRG5	Barang_Toko_5	Random5	9630
	BRG6	Barang_Toko_6	Random6	7060
	BRG7	Barang_Toko_7	Random7	2820
	BRG8	Barang_Toko_8	Random8	9620
	BRG9	Barang_Toko_9	Random9	9960
	BRG10	Barang_Toko_10	Random10	8280
	BRG11	Barang_Toko_11	Random11	3920
	BRG12	Barang_Toko_12	Random12	9030

Gambar 4.3 Menu Tampilkan Semua Data

SIJALAL

Sistem Inventaris Jalaludin

id barang

BRG54

nama barang

kategori

jumlah

harga

perbarui

tambah

cari

hapus

RESET

	ID Barang	Nama Barang	Kategori	Jumlah
▶	BRG54	Barang_Toko_54	Random54	97800
*				

Gambar 4.4 Menu Cari Barang

SIJALAL

Sistem Inventaris Jalaludin

id barang

nama barang

kategori

perbarui

Sukses

Barang dengan ID 'BRG54' dihapus!

OK

ID Barang	Nama Barang	Kategori	Jum
BRG1	Barang_Toko_1	Random 1	4200
BRG2	Barang_Toko_2	Random2	3350
BRG3	Barang_Toko_3	Random3	1700
BRG4	Barang_Toko_4	Random4	4790
BRG5	Barang_Toko_5	Random5	9630
BRG6	Barang_Toko_6	Random6	7060
BRG7	Barang_Toko_7	Random 7	2820
BRG8	Barang_Toko_8	Random8	9620
BRG9	Barang_Toko_9	Random9	9960
BRG10	Barang_Toko_10	Random10	8280
BRG11	Barang_Toko_11	Random11	3920
BRG12	Barang_Toko_12	Random12	9030

Gambar 4.5 Menu Hapus barang

Sistem Inventaris Jalaludin

id barang

nama barang

kategori

Sukses

 Data barang 'BRG55' sukses di - update!

ID Barang	Nama Barang	Kategori	Jum
BRG1	Barang_Toko_1	Random1	4200
BRG2	Barang_Toko_2	Random2	3350
BRG3	Barang_Toko_3	Random3	1700
BRG4	Barang_Toko_4	Random4	4790
BRG5	Barang_Toko_5	Random5	9630
BRG6	Barang_Toko_6	Random6	7060
BRG7	Barang_Toko_7	Random7	2820
BRG8	Barang_Toko_8	Random8	9620
BRG9	Barang_Toko_9	Random9	9960
BRG10	Barang_Toko_10	Random10	8280
BRG11	Barang_Toko_11	Random11	3920
BRG12	Barang_Toko_12	Random12	9030

Gambar 4.6 Menu Update

4.2 Kesesuaian Proyek dengan Laporan

1. Algoritma *Sorting*

Kami menjadikan *Algoritma Sorting* sebagai fitur untuk mengorganisir data inventaris secara otomatis. Dalam program ini, *Sorting* diterapkan secara spesifik untuk mengurutkan *ID_Barang*. Algoritma ini menyelesaikan masalah pencatatan yang berantakan, sehingga admin dapat memantau seluruh daftar barang yang masuk secara terstruktur dan sistematis tanpa harus menyusunnya secara manual.

2. Algoritma *Searching*

Untuk memecahkan masalah lambatnya pencarian stok di tengah banyaknya tumpukan data inventaris, kami mengimplementasikan algoritma *Searching*. Fitur ini menjadi kunci agar sistem mampu menemukan detail, kategori, atau ketersediaan suatu barang secara spesifik, instan, dan akurat hanya dengan memasukkan kata kunci pencarian.

3. **Operasi CRUD (*Create, Read, Update, Delete*)**

Agar algoritma *Sorting* dan *Searching* dapat memproses data yang dinamis, kami membangun fondasi operasi CRUD. Sistem ini memungkinkan pengguna untuk terus memanipulasi data inventaris (menambah barang baru, memperbarui detail, atau menghapus barang rusak) sebelum nantinya diurutkan dan dicari oleh algoritma utama.

4. **Tipe Data *Struct***

Sebagai wadah dari data yang akan diolah oleh algoritma, kami menggunakan *Struct Barang*. Materi ini digunakan untuk mengikat berbagai atribut data yang berbeda (ID, Nama Barang, Kategori, Stok, dan Harga) ke dalam satu entitas yang utuh agar lebih mudah dieksekusi saat proses pencarian dan pengurutan.

5. **Penyimpanan Data Menggunakan File .csv**

Seluruh data inventaris yang telah dikelola, diurutkan, dan dicari akan diekspor dan dibaca melalui file berekstensi *.CSV*. Hal ini memastikan perubahan terhadap data bersifat permanen, di mana data tidak akan hilang meskipun program telah ditutup.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Kadir, A. (2018). *Dasar Pemrograman C++*. Andi Offset.

Munir, R. (2011). *Algoritma dan Pemrograman dalam Bahasa Pascal dan C*. Informatika Bandung.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.

Sutabri, T. (2012). *Analisis Sistem Informasi*. Andi Offset.